

1. 프로젝트 개요

프로젝트명: JBNU-SHOPPING

개발 목적: 실제 쇼핑몰과 유사한 기능(회원 가입/로그인, 상품 등록, 장바구니, 주문, 할인, 쿠폰, 결제 등)을 구현하고, CI/CD, AWS ECS/Fargate 등 클라우드 환경에 대한 자동화 배포 경험을 쌓는 것이 핵심 목적입니다.

이 프로젝트는 데이터베이스 설계부터 서버 사이드 로직, 무중단 배포(블루-그린)까지 실제 상용 서비스와 유사한 **엔드 투 엔드(End-to-End)** 구조를 학습하고자 진행되었으며, Spring Boot를 기반으로 한 백엔드 구현과 AWS ECS/Fargate를 활용한 마이크로서비스형 배포 전략, GitHub Actions를 통한 CI/CD 자동화 환경 구축을 주요 골자로 삼았습니다.

2. 전체 아키텍처 및 구성

2.1 아키텍처 개요

1. 백엔드 (Spring Boot)

- RESTful API 서버 역할을 수행합니다.
- Spring Data JPA를 통해 MySQL 등 RDBMS와 연동하며, 회원/상품/주문/결제/할인 등 eCommerce 전반 로직을 관리합니다.

2. CI/CD 파이프라인 (GitHub Actions + AWS ECR + CodeDeploy)

- 코드 푸시 시 자동 빌드 및 Docker 이미지 생성 → AWS ECR에 업로드 → CodeDeploy를 통한 ECS Task 업데이트 → 무중단 배포(블루-그린 배포 방식) 순으로 진행됩니다.

3. AWS 인프라 (ECS/Fargate, EventBridge, Lambda 등)

- **ECS/Fargate**를 통해 EC2 인스턴스를 별도로 관리할 필요 없이 컨테이너 기반으로 애플리케이션을 실행합니다.
 - **EventBridge & Lambda**를 이용해 ECR 업데이트를 감지하고, 자동으로 CodeDeploy를 트리거하여 최신 버전 컨테이너를 배포합니다.
 - **AWS Secret Manager, VPC 분리** 등을 통해 보안 및 네트워크 안정성을 확보합니다.
-

3. 사용 기술 스택

- 언어/프레임워크:
 - Java 17, Spring Boot, Spring Data JPA
- 데이터베이스:
 - MySQL 또는 PostgreSQL (학습 및 상황에 따라 선택 가능)

- 빌드/의존성 관리:
 - Gradle
 - 테스트/검증:
 - JUnit, Mockito 등 (주요 테스트 시 사용)
 - CI/CD:
 - GitHub Actions, Docker, AWS ECR, CodeDeploy
 - 클라우드 및 인프라:
 - AWS ECS/Fargate, EventBridge, Lambda, Secret Manager, VPC, ELB(Load Balancer)
 - 기타:
 - Swagger(혹은 SpringDoc) 등을 통해 API 문서 자동화
 - Redis (세션 관리, 혹은 필요한 경우 캐싱용으로 활용 가능)
-

4. 도메인 모델 및 주요 엔티티 구조

JBNU-SHOPPING의 주요 기능(회원가입, 로그인, 상품 등록·조회, 장바구니, 주문/결제, 할인/쿠폰 등)에 따라 **도메인별**로 엔티티가 나뉩니다. 아래는 실제 코드 일부를 기반으로 한 요약입니다.

4.1 회원/인증 도메인

- **Account**
 - 사용자 정보를 담는 핵심 엔티티입니다.
 - 필드: `id`, `name`, `username`, `password`, `email`, `phoneNumber`, `role`, `point` 등
 - 다수의 `Address` (주소)를 가질 수 있습니다(일대다).
- **Address**
 - 회원이 가진 배송지 정보 엔티티입니다.
 - 필드: `zipcode`, `street`, `detail`, `addressType` 등
 - `AddressType` (`PRIMARY`, `SECONDARY`)을 통해 기본 주소/추가 주소를 구분합니다.

```
@Entity
public class Address extends BaseEntity {
    // ...
    @Enumerated(EnumType.STRING)
    private AddressType addressType;
    // ...
}
```

4.2 상품 도메인

- **Item**

- 쇼핑몰에서 판매되는 상품을 관리하는 엔티티입니다.
- 필드: `name`, `price`, `stockQuantity`, `itemImagePath`, `itemInfoImagePath`, `discount` (연관관계) 등
- `likeCount` 를 @Formula로 조회하여 상품 좋아요(Like) 수를 실시간으로 조회 가능하게 구성했습니다.
- 재고 수량 증가/감소 로직(`reduceStockQuantity`, `addStockQuantity`)과 할인 관련 로직을 포함합니다.
- **Discount**
 - 할인 정책 정보를 담는 엔티티입니다.
 - 필드: `name`, `discountPercent`, `startDate`, `endDate`, `discountStatus` 등
 - `Item` 과 연관관계를 맺어, 상품 단위로 할인 혜택을 적용할 수 있습니다.
- **Like**
 - 회원이 상품에 좋아요(찜)한 기록을 위한 엔티티입니다.
 - 필드: `account`, `item`, `likeStatus` (ACTIVE, INACTIVE)
 - @Formula가 반영된 `Item` 의 `likeCount` 와 연동됩니다.

4.3 주문 도메인

- **Order**
 - 회원(Account)이 상품(Item)을 주문하는 엔티티입니다.
 - 필드: `account`, `delivery` (Address), `coupon`, `orderDate`, `status` (ORDER, COMP, CANCEL) 등
 - `OrderItem` 과 일대다 관계를 맺으며, 주문 취소(`cancel`) 시 각 주문 상품의 재고를 복구하는 로직이 포함됩니다.
 - **OrderItem**
 - 한 번의 주문에 포함되는 개별 상품 정보를 저장합니다.
 - 필드: `item`, `order`, `orderPrice`, `discountPercent`, `count` 등
 - 주문 취소 시 `item.addStockQuantity(count)` 를 호출해 재고를 되돌려놓도록 구현되어 있습니다.
 - **Coupon**
 - 특정 회원(Account)에게 발급된 쿠폰 정보를 담습니다.
 - 필드: `name`, `startDate`, `endDate`, `minimumOrderAmount`, `discountAmount` 등
 - 주문 시 쿠폰이 존재하면 할인 금액을 적용할 수 있도록 구성합니다.
 - **CartItem**
 - 장바구니에 담긴 상품을 관리하는 엔티티입니다.
 - 필드: `account` (어떤 회원의 장바구니인지), `item`, `quantity` 등
 - 수량 증가/감소(`updateQuantity`)가 가능하며, Order로 넘기는 과정에서 `OrderItem` 으로 전환됩니다.
-

5. 주요 기능 및 구현 상세

5.1 회원가입 및 인증

- **Spring Security** (또는 JWT) 기반 로그인 기능을 구현할 수 있도록 설계되었습니다.
- Account 를 생성하고, 비밀번호 암호화(BCrypt 등) 과정을 거쳐 DB에 저장합니다.
- 로그인 성공 시 JWT 발급(또는 세션 관리)로 인증/인가를 처리합니다.

5.2 상품 관리

- 관리자(ROLE_ADMIN)는 Item 을 등록/수정/삭제할 수 있습니다.
- 할인을 진행할 때는 Discount 엔티티를 생성하여 특정 기간(startDate , endDate) 동안 Item 가격에 할인율을 적용합니다.

5.3 장바구니 및 주문

1. 장바구니

- 회원이 원하는 상품을 CartItem 엔티티로 생성해 장바구니에 담습니다.
- 담긴 상품 수량을 증감하거나 제거할 수 있습니다.

2. 주문

- CartItem 에 있던 상품 정보를 이용해 Order 와 OrderItem 을 생성합니다.
- 재고(quantity) 부족 시 NotEnoughStockQuantityException 을 발생시키도록 하여 안정성을 높였습니다.
- 결제 완료 시 OrderStatus 를 COMP 로 변경하여 주문을 완료하고, 이후 변경 불가능하도록 제어합니다.

5.4 쿠폰 및 할인

- Coupon 엔티티를 통해 회원이 일정 금액 이상 구매 시 일정 금액을 할인받는 등 다양한 형태의 쿠폰을 적용할 수 있습니다.
- Discount 엔티티는 Item 단위 할인 정보를 저장하여 상품 자체의 할인율을 제공할 수 있습니다.

5.5 CI/CD 자동화 및 무중단 배포

1. GitHub Actions

- push 또는 pull_request 이벤트 발생 시 백엔드/프론트엔드 각각 Docker 이미지로 빌드 → AWS ECR 업로드
- 이전에는 직렬 작업으로 5분 이상 걸리던 빌드 과정을 병렬화해 2분대로 단축

2. AWS ECS/Fargate + CodeDeploy (블루-그린 배포)

- ECR에 새로운 이미지가 업데이트되면 EventBridge가 이를 감지하고, Lambda가 CodeDeploy를 트리거
- 무중단 배포로 서비스 중단 없이 배포가 가능하며, 배포 실패 시 자동 롤백 지원

6. 운영 환경 및 이슈 해결 사례

1. 환경 변수 주입 이슈

- AWS Secret Manager에서 환경 변수를 가져올 때 ECS Task 정의 설정 오류로 인해 컨테이너가 무한 재시작
- Task Definition 내에서 정적 환경 변수를 설정하고, Secret Manager 키를 정확히 지정하여 해결

2. 헬스 체크 문제

- 기본 경로("/")로 헬스 체크 시 Swagger 페이지가 404를 반환해 배포 실패 발생
- 배포 성공을 위해 실제 동작하는 엔드포인트(예: /api/health 등)로 헬스 체크 경로를 변경

3. ECR 분리

- 프론트엔드, 백엔드 이미지를 단일 ECR에서 태그로만 구분했을 때 충돌 발생
- 서로 다른 리포지토리로 분리하여 태그 충돌 문제 해결, 배포 시 혼선 방지

7. 프로젝트 성과 및 확장성

1. 무중단 배포 체계

- CI/CD 전 과정을 자동화함으로써 빌드·배포 시간을 단축하고, 운영 중단 없이 서비스를 배포할 수 있게 되었습니다.

2. 실제 서비스와 유사한 기능 구현

- 사용자 인증, 재고 관리, 쿠폰/할인 기능, 장바구니, 주문 등 이커머스의 주요 기능을 골고루 포함했습니다.

3. 문제 해결 능력 및 DevOps 역량

- 도커 컨테이너를 기반으로 한 클라우드 운영, 네트워킹(Load Balancer, VPC, 보안 설정), 이벤트 드리븐 배포 자동화 등
- 실무에서 빈번히 마주치는 “환경 변수 설정 오류”, “헬스 체크 실패” 등의 배포 이슈를 직접 해결하며 운영 노하우를 쌓았습니다.

4. 확장성

- 마이크로서비스 아키텍처(MSA)로 확장하거나, Kubernetes(K8s) 같은 오케스트레이션 도구로 이관하는 등 추가적인 인프라 고도화에 적합하도록 설계되었습니다.
- 대용량 트래픽 대응을 위해 Redis, 메시지 큐(RabbitMQ, Kafka) 등을 적용해볼 수 있는 구조입니다.

8. 결론

JBNU-SHOPPING은 회원 인증부터 상품 할인, 장바구니 및 주문, 쿠폰 발행, 무중단 배포(블루-그린)까지 **상용 수준 쇼핑몰 서비스를 체험**해볼 수 있도록 설계된 프로젝트입니다.

- **도메인 모델**을 명확히 분리해, **확장성과 유지보수성**을 높였으며,
- **CI/CD 자동화, AWS ECS/Fargate** 기반 배포로 **DevOps 역량**을 실제 환경에서 다뤄볼 수 있도록 구성했습니다.

이 프로젝트를 통해 **백엔드 코어 로직**(재고 관리, 할인 정책, 주문 처리, 예외 처리)을 설계·구현하는 능력뿐 아니라, **운영 환경**에서 발생하는 배포 이슈나 보안 설정 문제를 직접 해결하며 성장할 수 있었습니다. 곧바로 상용화할 수준의 구조는 아니더라도, **실전 프로젝트**에서 자주 필요한 기능들을 빠짐없이 담고 있고, **추가 스케일업**(컨테이너 오케스트레이션, 글로벌 트래픽 대응)에도 유연하게 대처할 수 있는 좋은 학습 자료가 됩니다.

JBNU-SHOPPING 프로젝트의 개요부터 핵심 기능, 배포 환경, 성과까지 전반적인 내용을 정리했습니다. 실제 운영 시나리오와 유사하게 **개발-테스트-배포** 전 과정이 유기적으로 연결되어 있어, 팀 협업 및 실무 역량 향상에 큰 도움이 될 수 있는 프로젝트입니다.